

Programare Orientata Pe Obiecte

Curs 11 - Expresii Lambda

Curs 12 - UML, Java IO

Decembrie 2025

Agenda

- 1 Clase Anonime si Expresii Lambda
- 2 UML
- 3 Studiu de caz

Clase Anonime si Expresii Lambda

Exemplu

- ❶ Consideram o aplicatie JAVA care gestioneaza cartile dintr-o biblioteca
- ❷ O carte este caracterizata de:
 - Titlu (String)
 - Autor (Persoana)
 - Tip (ARTA, BIOGRAFIE, FICTIUNE, DEZVOLTARE_PERSONALA)
 - Pret (double)
 - Numar de pagini
- ❸ CatalogCarti - contine lista de carti
- ❹ Se cere sa se afiseze cartile dupa anumite criterii, de exemplu:
 - Toate cartile de tip biografie
 - Toate cartile cu mai putin de 100 de pagini
 - Toate cartile de tip fictiune cu pret mai mic de 30 lei.

Exemplu 1: Afiseaza toate cartile de tip BIOGRAFIE

```
1 public class Main {  
2     public static void main(String[] args) {  
3         List<Carte> carti = CatalogCarti.initializeazaCatalog();  
4         afiseazaCartiBIOGRAFICE(carti);  
5     }  
6     private static void afiseazaCartiBIOGRAFICE(List<Carte> carti) {  
7         for (Carte c:carti)  
8             if (c.getTip() == TipCarte.BIOGRAFIE)  
9                 System.out.println(c);  
10    }  
11 }
```

La linia 8 se verifica tipul cartii.

Dar dacă dorim să afișăm toate cărțile de tip FICTIUNE ?

Putem scrie o metodă nouă sau să schimbăm metoda la afiseazaCarti cu un parametru care sa specifice tipul cărții.

Parametrizarea comportamentului metodelor

- Dacă dorim să modificăm metoda ca să afișeze cărți mai ieftine de 30 lei ?
 - scriem o altă metodă ?
 - adăugăm criterii suplimentare și argumente în plus la metodă ca să specificăm cu ce criterii se face afișarea ?
- Cum acoperim multiple combinații și criterii de afișare ?
- Codul pentru aceste metode ar fi cam la fel pentru că:
 - Traversarea listei și afișarea e la fel
 - Se schimbă doar criteriul de comparație
- Cum putem să parametrizăm criteriul de comparație care se schimbă?

Java 8: Predicate și Interfețe Funcționale

- În Java 8, au fost introduse interfețele funcționale, care reprezintă interfețe cu o singură metodă neimplementată / abstractă (dar pot avea metode default sau statice).
 - folosim adnotarea `@FunctionalInterface`, prin care se permite existența unei singure metode abstracte în cadrul unei interfețe funcționale.
 - se folosesc pentru expresii lambda
- Interfețe funcționale predefinite:
 - *Predicate* $\langle T \rangle$ - cu metoda abstractă *boolean test(T t)*
 - *Consumer* $\langle T \rangle$ - cu metoda abstractă *accept(T t)*

Interfața Predicate: Exemplu 2

Definim predicatele în clase separate:

```
1 import java.util.function.Predicate;
2 public class CarteIeftina implements Predicate<Carte> {
3     @Override
4     public boolean test(Carte carte) {
5         return carte.getPret() < 30;
6     }
7 }
```

```
1 import java.util.function.Predicate;
2 public class CarteBiografica implements Predicate<Carte> {
3     @Override
4     public boolean test(Carte carte) {
5         return carte.getTip() == TipCarte.BIOGRAFIE;
6     }
7 }
```

Folosirea predicatelor

Modificam metoda de afişare a cărţilor ca să accepte predicate de care depinde dacă se afişează cărţile sau nu:

```
1  import java.util.List;
2  import java.util.function.Predicate;
3  public class Main {
4      public static void main(String[] args) {
5          List<Carte> carti = CatalogCarti.initializeazaCatalog();
6          System.out.println("Carti ieftine:");
7          afiseazaCarti(carti, new CarteIeftina());
8          System.out.println("Carti biografice:");
9          afiseazaCarti(carti, new CarteBiografica());
10     }
11     private static void afiseazaCarti(Iterable<Carte> carti,
12                                     Predicate<Carte> pr) {
13         for (Carte c : carti) {
14             if (pr.test(c)) System.out.println(c);
15         }
16     }
```

Avantajele utilizării predicatelor

Redundanța codului:

- Nu mai avem multe metode cu aproximativ același cod
- Schimbarea sau adăugarea de criterii (predicate) noi nu modifică metoda afiseazaCarti.
- Refolosirea codului:
 - Metoda "deșteaptă" poate fi exportată iar utilizatorul pregătește alte predicate

Clase anonime și interfețe funcționale: Exemplu 3

Predicatele din Exemplul 2 au fost definite în clase separate și au fost folosite doar o dată (posibil să nu le mai folosim ulterior).

- Folosim clase anonime pentru aceste situații
- Declarăm și instanțiem o clasă în același timp
- Clasa nu are nume (este anonimă)
- Clasa extinde o altă clasă sau implementează o interfață

```
1 public static void main(String[] args) {  
2     List<Carte> carti = CatalogCarti.initializeazaCatalog();  
3     System.out.println("Carti biografice:");  
4     afiseazaCarti(carti, new Predicate<Carte>() {  
5         @Override  
6         public boolean test(Carte c) {  
7             return c.getTip() == TipCarte.BIOGRAFIE;  
8         }  
9     });  
10 }
```

Clase anonime: proprietăți

- Clasele anonime fie extind o altă clasă (clasa părinte nu e neapărat abstractă) sau implementează o interfață
 - Dacă este extinsă o clasă abstractă sau o interfață atunci toate metodele abstracte trebuie implementate (deoarece deodată cu clasa anonimă se creează și un obiect)
 - Clasa anonimă poate avea metode adiționale sau câmpuri (variabile).
- Clasele anonime nu au nume, deci nu vor avea constructori, dar pot avea blocuri de inițializare.

Clasa anonimă care extinde o altă clasă: exemplul 4

- Avem o clasa care filtrează cărțile pe baza prețului:

```
1 import java.util.function.Predicate;
2
3 public class PredicatPretCarte implements Predicate<Carte> {
4     private int pretMaxim;
5     public PredicatPretCarte(int pretMaxim) {
6         this.pretMaxim = pretMaxim;
7     }
8     @Override
9     public boolean test(Carte c) {
10         return c.getPret() < pretMaxim;
11     }
12 }
```

Clasa anonimă care extinde o altă clasă: exemplul 4

- Clasa anonimă extinde clasa predicat definita anterior
- Suprascrie metoda test cu criterii suplimentare de filtrare (Se vor afisa cartile cu pret < 50 si numar pagini > 400)

```
1 import java.util.function.Predicate;
2 public static void main(String[] args) {
3     List<Carte> carti = CatalogCarti.initializeazaCatalog();
4     afiseazaCarti(carti, new PredicatPretCarte(50){
5         int numarPagini;
6         {
7             numarPagini = 400;
8             System.out.println("Init numarPagini: " + numarPagini)
9         }
10    @Override
11    public boolean test(Carte c) {
12        return super.test(c) && c.getNrPagini() > numarPagini;
13    } } );
14 }
```

- Parametrul 50 de pe linia 4 va initializa pretMaxim din clasa PredicatePretCarte
- De la linia 6 pana la 9 avem un bloc de initializare static.

Clase anonime: observații

- Într-o clasă anonimă `this` refera obiectul clasei anonime
- Folosim clase interne anonime în locul definirii unei clase cu număr de utilizări reduse
- Clasele anonime nu pot avea constructori din cauză că nu au nume (nu am ști cum să numim constructorii).
- Această restricție asupra claselor anonime ridică o problemă: în mod implicit, clasa de bază este creată cu constructorul default.
- O clasă internă anonimă poate extinde o clasă sau să implementeze o singură interfață, nu poate face pe ambele împreună ca la clasele ne-anonime (interne sau nu), și nici nu poate să implementeze mai multe interfețe.

Expresii Lambda

- Lambda este un concept din programarea funcțională și reprezintă o funcție anonimă.
- Majoritatea limbajelor de nivel înalt au introdus suport pentru acest concept, inclusiv Java, din versiunea 8.
- În exemplul următor am transformat clasa anonimă din secțiunea precedentă în expresie lambda.

```
1 import java.util.function.Predicate;
2 public static void main(String[] args) {
3     List<Carte> carti = CatalogCarti.initializeazaCatalog();
4     afiseazaCarti(carti, (c)->{return c.getPret() < 35 &&
5         c.getNrPagini() > 400;});
6     afiseazaCarti(carti, (c)->{return c.getTip()==TipCarte.BIOGRAFIE
7         ;});
8 }
9 private static void afiseazaCarti(Iterable<Carte> carti,
10     Predicate<Carte> pr) {
11     for (Carte c : carti) {
12         if (pr.test(c)) System.out.println(c);
13     }
14 }
```

Expresii Lambda

- Al doilea argument al metodei afiseazaCarti este:
 - `(c)->{return c.getPret() < 35 &&
 c.getNrPagini() > 400;}`
 - `(c)->{return c.getTip() == TipCarte.BIOGRAFIE;}`
- Cum interpretăm codul?
 - Compilatorul "citește" semnătura metodei afiseazaCarti si determină că al doilea argument așteptat este un obiect de tipul *Predicate < Carte >*
 - Predicate este o interfață funcțională cu o metodă abstractă care primește ca parametru un obiect de tip Carte și returnează un boolean
 - În partea stângă a săgeții sunt parametrii metodei funcționale (tipul poate fi omis)
 - În partea dreaptă a săgeții este valoarea returnată

Interfața funcțională *Consumer* < *T* >

- În exemplele anterioare am afișat cărțile care îndeplineau anumite condiții.
- Se poate face codul mai abstract și să lăsăm acțiunea de afișare celor care utilizează metoda
- În loc de afișare se efectuează o metodă (adică obiectul este **consumat**)
- Se folosește interfața funcțională *Consumer* < *T* > cu metoda funcțională: **void accept(T t)**

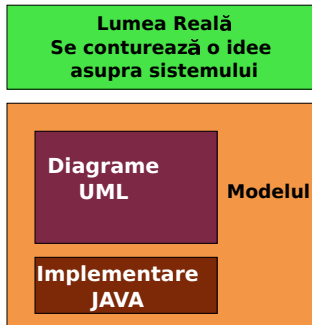
Exemplu cu interfața *Consumer* < *T* >

```
1 import java.util.function.Consumer;
2 import java.util.function.Predicate;
3 private static void afiseazaCarti(Iterable<Carte>carti,
4     Predicate<Carte> pr, Consumer<Carte> actiune){
5     for (Carte c:carti){
6         if (pr.test(c)) actiune.accept(c);
7     }
8 }
9 public static void main(String[] args){
10     List<Carte> carti = CatalogCarti.initializeazaCatalog();
11     afiseazaCarti(carti,
12         (c)->c.getPret() < 30,
13         (c)->System.out.println("Carte cu pret sub 30 lei: " + c));
14     afiseazaCarti(carti,
15         (c)->c.getTip() == TipCarte.BIOGRAFIE,
16         (c)->System.out.println("Carte de tip BIOGRAFIE: " + c));
17 }
```

UML

UML - Unified Modified Language

- UML este notația internațională standard pentru analiza și proiectarea orientată pe obiecte
- Sunt definite mai multe tipuri de diagrame, împărțite în două categorii:
 - 1 Diagrame de structură (6): descriu structura statică a aplicației
 - 2 Diagrame de comportament(7): descriu tipuri generale de comportament



UML - tipuri de diagrame

Diagrame de structură:

- Diagrama de clase
- Diagrame de obiecte
- Diagrama de componente
- Diagrama de structură compozită
- Diagrama de pachete
- Diagrama de desfășurare sistematică

Diagrame de comportament:

- Diagrama de activități
- Diagrama de interacțiune
- Diagrama cazurilor de utilizare (use case)
- Diagrama de secvențe
- Diagrama de stare
- Diagrama de comunicare
- Diagrama de timp

Diagrame de clasă

Modelează privirea statică, definesc vocabularul si cuprind:

- Clase
 - Clasa este o colecție de obiecte asemănătoare
 - Clasa este definită de atribute și de operații (metode)
- Relatiile intre clase:
 - Asociere: poate fi agregare sau compoziție
 - Generalizare
 - Dependență
 - Realizare (sau implementare)

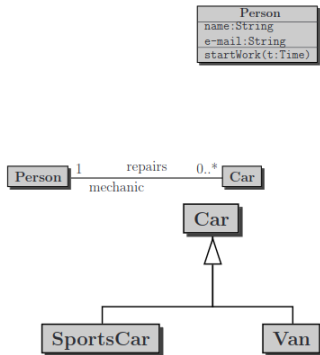


Diagrama de clase - Exemplu 2

- Reprezintă un set de clase, interfețe, colaborări și alte relații
- Reflectă proiectul static al unui sistem
- Poate genera confuzii dacă este folosit pentru a explica dinamica sistemului; folosiți în loc diagramele de obiecte care sunt mai puțin abstracte

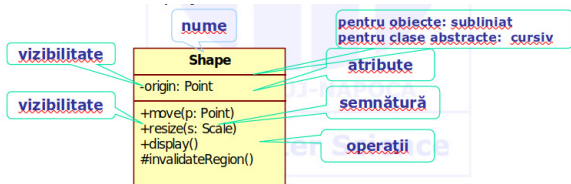
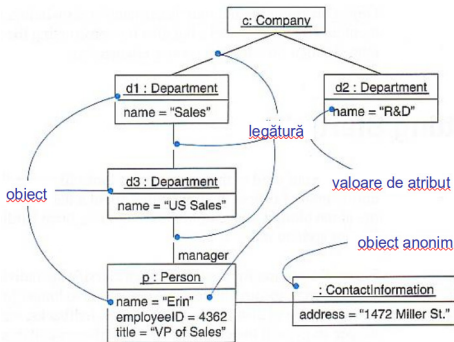


Diagrama de obiecte

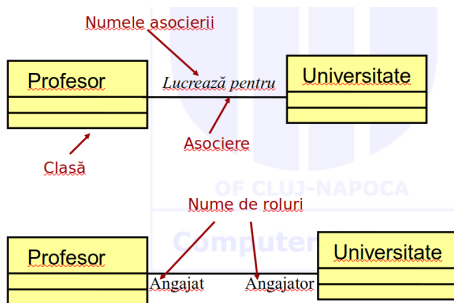
- Reprezintă un set de obiecte (instanțe de clase) și relațiile dintre acestea
- Oferă o vedere dinamică a sistemului la un moment dat
- Reprezintă cazuri reale sau cazuri prototip
- Foarte utile înainte de dezvoltarea diagramelor de clase
- Diagrama de obiecte alăturată este o instanțiere a diagramei de clase (clasele fiind înlocuite de obiecte concrete)



Relații între clase

- Asocierie: poate fi agregare sau compoziție
- Generalizare
- Dependență
- Realizare (sau implementare)

Asocierea: modelează o conexiune semantică între clase



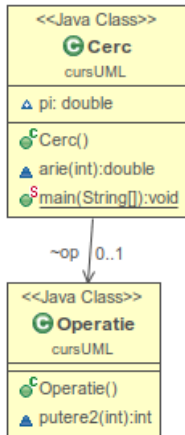
Folosirea asocierilor

- ❶ Pentru a reprezenta o situație în care un obiect de o clasă folosește serviciile unui alt obiect, sau ele își folosesc reciproc serviciile – adică un obiect îi trimite mesaje celuilalt sau își trimit mesaje între ele
 - În primul caz, navigabilitatea poate fi unidirecțională; în cel de al doilea, ea trebuie să fie bidirecțională
- ❷ Agregarea sau compoziția – unde obiecte de o clasă sunt întregi compuși din obiecte de cealaltă clasă ca părți
 - În acest caz, o relație de tip “folosește” este implicit prezentă – întregul folosește părțile pentru a-și îndeplini funcția, iar părțile pot și ele avea nevoie să folosească întregul
- ❸ O situație în care obiectele sunt înrudite, chiar dacă nu schimbă mesaje
 - Aceasta se întâmplă de obicei când cel puțin unul dintre obiecte este folosit în esență la stocare de informație

Asociere exemplu

O clasa foloseste serviciile altei clase:

```
1 public class Operatie {
2     int putere2(int n) {return n*n;}
3 }
4 public class Cerc {
5     Operatie op;
6     double pi = 3.14;
7     double arie(int r) {
8         op = new Operatie();
9         int r2 = op.putere2(r);
10        return pi*r2;
11    }
12    public static void main(String[] args) {
13        Cerc c = new Cerc();
14        double res = c.arie(5);
15        System.out.println("arie cerc=" + res);
16    }
17 }
```



Asociere: Multiplicitate și navigare

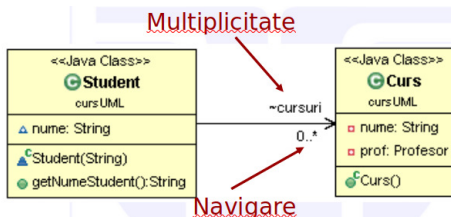
- Multiplicitatea definește câte obiecte participă într-o relație
- Numărul de instanțe ale unei clase în raport cu o instanță a celeilalte clase
- Specificat pentru fiecare capăt al asocierii
- Asocierile sunt implicit bidirecționale, dar adesea este de dorit să se restrângă navigarea la o singură direcție
- Dacă navigarea este restricționată, se adaugă o săgeată pentru a indica direcția de navigare

Asociere: multiplicitate

Nespecificată	_____	
Exact una	1	_____
Zero sau mai multe (multe, nelimitat)		0..* _____
Una sau mai multe	1..*	
Zero sau una	0..1	
Gama specificată	2..4	
Game multiple, disjuncte	2, 4..6	

Exemplu: multiplicitate si navigare

```
1 public class Curs {
2     private String nume;
3     private Profesor prof;
4 }
5 public class Student {
6     String nume;
7     Curs[] cursuri;
8     Student(String n) {
9         nume = n;
10        cursuri = new Curs[5];
11    }
12    public String getNumStudent() { return nume; }
13 }
```

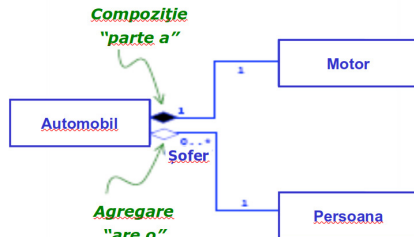


Asociere: agregare și compoziție

Asocierea poate fi :

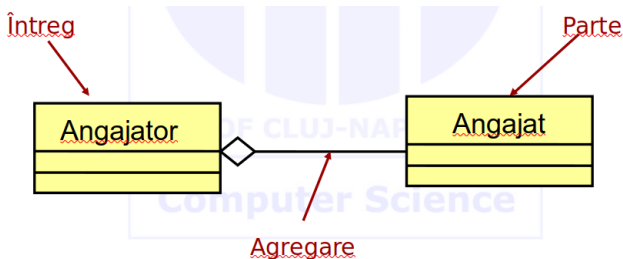
- Agregare sau
- Compoziție

În ambele cazuri obiecte de o clasă sunt întregi compuși din obiecte de cealaltă clasă ca părți.



Agregare

- Relație de tip: “are o/un” HAS-A
- O formă specială de asociere care modelează relația parte-întreg între un agregat (întregul) și părțile sale



Agregare exemplu

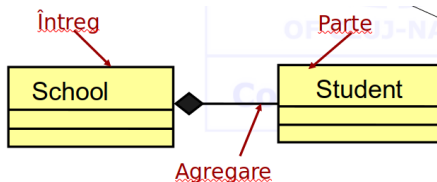
```
1 public class Student {
2     String nume;
3     Student(String n){ nume = n; }
4     public String getNumStudent() {return nume;}
5 }
6 public class School {
7     private String name;     private Student stud;
8     School(String n){ name = n; }
9     public void setStudent(Student s) {stud = s;}
10    public String getName() { return name; }
11 }
12 public class StudentDemo {
13 public static void main(String[] args) {
14     School s = new School("UTCN");
15     Student a = new Student("Victor");
16     s.setStudent(a);
17 }
```

Chiar daca “stergem” obiectul s de la linia 14, studentul instantiat la linia 15 va continua să existe pentru că este creat în afara clasei School si este transmis ca parametru unei metode din clasa School !

Compoziție

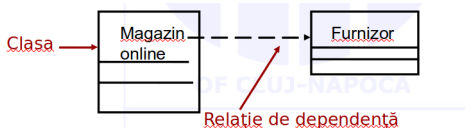
- Relatie de tip: “este parte a”
- O formă de agregare cu posesiune puternică și durate de viață care coincid
- Părțile nu pot supraviețui fără existența întregului/ agregatului
- Obiectul e creat in interiorul metodei unei clase

```
1 public class School {  
2     private String name;  
3     private Student stud;  
4     School(String n){  
5         name = n;  
6     }  
7     public void setStudent() {  
8         stud = new Student("Maria");  
9     }  
10    ...  
11 }
```



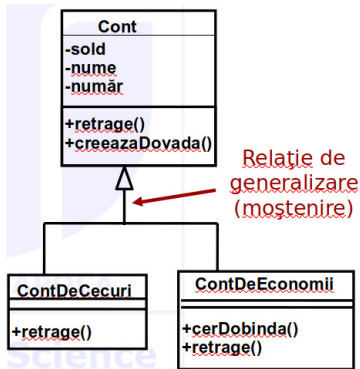
Dependență

- Relație de tip “folosește”
- O relație între două elemente ale modelului în care o schimbare în unul dintre elemente poate cauza o schimbare în celălalt



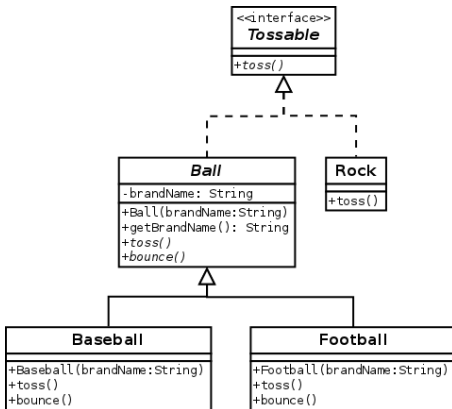
Generalizare sau moștenire

- Relație de tipul este-o/un
- Relație între o clasă mai generală (superclasă) și una mai specializată (subclasă)
- Exemple:
 - Orice cont de economii este un cont bancar
 - Orice cerc este o elipsă (cu lățime și înălțime egale)



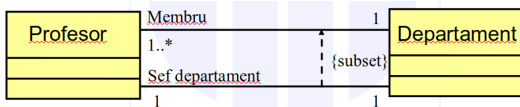
Realizare sau implementare

- O interfață servește pe post de contract pe care celălalt este de acord să-l îndeplinească
- Regăsit între interfețele și clasele care le realizează (implementează)



Constrângeri

Suportă adăugarea de reguli noi sau modificarea regulilor existente.



- Această notație este folosită pentru a surprinde două relații între obiecte de tip Profesor și obiecte de tip Catedra, unde una dintre relații este un subset al celeilalte
- Arată cum se pot croi diagramele UML pentru a modela corect relații exacte

Simboluri UML pentru relații

<u>Relație</u>	Simbol	Stil de linie	Forma vârfului
Moștenire		Solid	Triunghi
Implementare de interfață		Întrerupt	Triunghi
Agregare		Solid	Romb
Dependență		Întrerupt	Deschisă
Compozitie		Solid	Romb umplut
Asociere		Solid	

Studiu de caz

Mini studiu de caz: Biblioteca (1/3)

- 1 Considerăm o aplicație Java care gestionează cărțile și împrumuturile dintr-o bibliotecă.
- 2 O **carte** este caracterizată de:
 - Titlu (`String`)
 - Autor (o **Persoană**)
 - Tip (`ARTA`, `BIOGRAFIE`, `FICTIUNE`, `DEZVOLTARE_PERSONALA`)
 - Preț (`double`)
 - Număr de pagini (`int`)
- 3 **CatalogCarti** conține lista de cărți disponibile în bibliotecă.
- 4 Utilizatorii se pot înscrie ca **MembriBiblioteca** și pot împrumuta cărți.

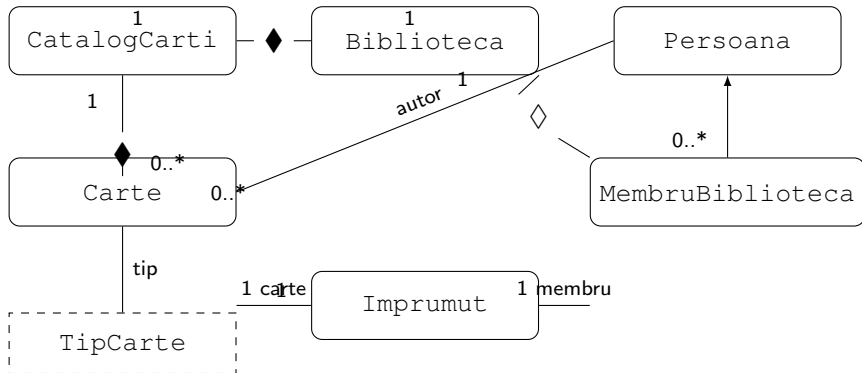
Mini studiu de caz: Biblioteca (2/3) – Modelare UML

Se dorește modelarea aplicației folosind UML.

Cerințe de analiză:

- 1 Identificați clasele principale:
 - Carte, Persoana, MembruBiblioteca, CatalogCarti, Imprumut, Biblioteca
- 2 Stabiliți atributele esențiale pentru fiecare clasă (ex.: nume, email pentru Persoana, date pentru Imprumut etc.).
- 3 Desenați **diagrama de clase UML** care să evidențieze:
 - relația dintre Biblioteca și CatalogCarti
 - relația dintre CatalogCarti și Carte
 - relația dintre MembruBiblioteca și Imprumut
 - relația dintre Imprumut și Carte
- 4 Marcați tipul relațiilor: asociere, agregare, compoziție, moștenire, dependență.

Diagrama de clase – Mini studiu de caz Biblioteca



Mini studiu de caz: Biblioteca (3/3) – Cerințe funcționale

Aplicația trebuie să permită:

- Adăugarea/ștergerea de cărți din `CatalogCarti`.
- Căutarea și afișarea cărților după criterii:
 - toate cărțile de tip `BIOGRAFIE`
 - toate cărțile cu mai puțin de 100 de pagini
 - toate cărțile de tip `FICTIUNE` cu preț mai mic de 30 lei
- Înregistrarea de **membri** ai bibliotecii (`MembruBiblioteca`).
- Înregistrarea unui **împrumut** (`Imprumut`) pentru un membru:
 - carte împrumutată, data împrumutului, data scadentă, status (returnată / nu).
- Afișarea tuturor împrumuturilor active pentru un anumit membru.

Exercițiu: extindeți diagrama de clase UML pentru a include `MembruBiblioteca` și `Imprumut`.

Mini studiu de caz – Întrebări UML

Pe baza modelului bibliotecii:

- ❶ Ce tip de relație este mai potrivit între:
 - CatalogCarti și Carte: agregare sau compoziție?
 - Biblioteca și CatalogCarti?
 - MembruBiblioteca și Imprumut?
- ❷ Ce multiplicități ați pune între:
 - CatalogCarti și Carte (1..*, 0..*, etc.)
 - MembruBiblioteca și Imprumut (1..*, 0..1, etc.)

Ce se întâmplă cu împrumuturile dacă ștergem un membru? Dar dacă ștergem o carte din catalog?

Mini studiu de caz – UML către Java

Pornind de la diagrama de clase, un posibil schelet de cod:

```
1  enum TipCarte {
2      ARTA, BIOGRAFIE, FICTIUNE, DEZVOLTARE_PERSONALA
3  }
4
5  class Persoana {
6      private String nume;
7      private String email;
8  }
9
10 class MembruBiblioteca extends Persoana {
11     private String nrLegitimatie;
12 }
13
14 class Carte {
15     private String titlu;
16     private Persoana autor;
17     private TipCarte tip;
18     private double pret;
19     private int nrPagini;
20 }
```

Mini studiu de caz – UML către Java (2)

```
1 class Imprumut {
2     private Carte carte;
3     private MembruBiblioteca membru;
4     private LocalDate dataImprumut;
5     private LocalDate dataScadenta;
6     private boolean returnata;
7 }
8
9 class CatalogCarti {
10     private List<Carte> carti;
11
12     List<Carte> cautaDupaTip(TipCarte tip) { ... }
13     List<Carte> cautaDupaNrPaginiMax(int maxPagini) { ... }
14     List<Carte> cautaFictiuneIeftina(double pretMax) { ... }
15 }
```

Diagrame cazuri de utilizare (Use Case)

- O **diagramă a cazurilor de utilizare** descrie **ce face** sistemul, văzut din perspectiva utilizatorilor externi.
- Elementele principale:
 - **Actor** – rol extern care interacționează cu sistemul
 - ex.: utilizator, membru bibliotecă, bibliotecar
 - **Caz de utilizare (use case)** – funcționalitate oferită actorilor
 - ex.: "Caută cărți", "Împrumută carte"
 - **Boundary** (limita sistemului) – conturul aplicației modelate
- Se folosesc pentru:
 - a discuta cu "clientul" ce trebuie să facă sistemul
 - a identifica funcționalitățile majore înainte de detalii de clasă / cod

Biblioteca – Diagrama cazurilor de utilizare

Studiu de caz: Aplicația de bibliotecă permite:

- membrilor să caute cărți și să le împrumute / returneze
- bibliotecarului să gestioneze catalogul de cărți

